

Talking to the OS.

`subprocess` · `osascript` bridge.

Python stops staying in the script. It opens apps, runs shell, fires AppleScript.

Yesterday: decorators. Today: the bridge.

Decorators are wrappers — extra work around a function. Today's tools are also wrappers, but around something bigger: the operating system itself. Python becomes a remote control.

✓ DONE · DAY 15

Decorators + functional patterns

- @decorator syntax = func = wrap(func)
- *args / **kwargs for forwarding
- functools.wraps preserves metadata
- @timer, @retry(n), @log_calls
- Lambda, map, filter quick tour

→ TODAY · DAY 16

Python drives the OS

- subprocess.run — shell from Python
- Capture stdout, check returncode
- shell=True vs list-of-args (safety)
- open -a → launch macOS apps
- osascript -e → AppleScript one-liners

Why this now? Every Week 3 tool — hotkey launcher, Safari opener, Spotify control — sits on top of subprocess. Learn the bridge first.

Run any command-line program from Python.

subprocess is Python's way of starting another program (like `ls`, `open`, `say`, `osascript`) and getting its result back. If you can type it in Terminal, subprocess can run it.

```
import subprocess

# run `ls` in the current directory
result = subprocess.run(["ls", "-l"],
                        capture_output=True,
                        text=True)

print(result.stdout)    # the text output
print(result.returncode) # 0 = success
```

ANATOMY

subprocess.run([...])

Pass the command as a list. First item is the program, rest are its arguments.

capture_output=True

Without this, output goes straight to Terminal and Python never sees it.

text=True

Gives you a string. Without it, you get raw bytes and need to `.decode()`.

result.stdout, result.stderr, result.returncode.

Every run() gives back a CompletedProcess object with three things you care about: what the program printed, what errors it printed, and whether it succeeded.

```
result = subprocess.run(["whoami"],
                        capture_output=True, text=True)

print("user:  ", result.stdout.strip())
print("errors: ", result.stderr)
print("status: ", result.returncode)
```

OUTPUT

```
user:  yuvraj
errors:
status: 0
```

WHAT YOU GET BACK

.stdout

The normal text output. Often ends with \n — call .strip() to clean it.

.stderr

Error messages. Empty string on success.

.returncode

0 = success. Anything else = the command failed. Check this before trusting stdout.

Pass a list. Avoid shell=True unless you really need it.

Two ways to give subprocess a command. The list form is safer and more predictable.

shell=True lets you write a full shell string — pipes, redirects, wildcards — but opens you up to injection if any part comes from user input.

RISKY · shell=True

```
name = input()
subprocess.run(f"echo Hello {name}",
               shell=True)
```

*# if name = `'; rm -rf ~`
the shell runs BOTH commands.*

SAFE · list of args

```
name = input()
subprocess.run(["echo", "Hello", name])
```

*# `name` is passed as ONE argument.
Shell metachars stay inert.*

Rule of thumb: default to the list form. Reach for shell=True only when you need pipes (`a | b`) or wildcards (`\*.py`).

open -a 'AppName' — your Python-side app launcher.

macOS ships with a command-line tool called `open` that launches apps, files, and URLs.

From Python, you just call it through subprocess. This is how the Morning Launcher will start Safari, Spotify, Notes, anything.

```
import subprocess

def launch(app_name):
    """Open a macOS app by name."""
    subprocess.run(["open", "-a", app_name])

launch("Safari")
launch("Spotify")
launch("Notes")

# open a URL directly:
subprocess.run(["open", "https://duke.edu"])
```

THE open COMMAND

open -a AppName

Launch the named app. The -a flag means 'by application name'.

open file.pdf

Open a file in its default app. No -a needed.

open https://...

Open a URL in your default browser.

osascript -e → Python controls macOS apps.

AppleScript is Apple's own scripting language for controlling Mac apps — opening Safari tabs, telling Spotify what to play, posting notifications.

You don't write AppleScript files. You hand a one-line script to the `osascript` tool and run it through subprocess.

```
import subprocess

def osa(script):
    """Run one AppleScript line."""
    return subprocess.run(["osascript", "-e", script],
                          capture_output=True, text=True)

# notification banner
osa('display notification "Build done" with title "CRUX"')

# open a new Safari tab
osa('tell application "Safari" to open location "https://duke.edu"')
```

WHY -e

The -e flag lets you pass the script as a string, instead of saving it to a .scpt file. Perfect for one-liners.

WHAT IT UNLOCKS

- Native macOS notifications
- Safari / Chrome tab control
- Spotify play / pause / next
- System volume, brightness
- Send Messages / Mail
- Move + resize app windows

Pick the right tool for the job.

All three flow through subprocess.run, but the command you hand it changes what you can do.

TOOL	USE FOR	EXAMPLE	WHEN
Plain shell cmd	Quick info, file ops	ls, whoami, date, pwd	Reading system state
open -a	Launching apps + URLs	open -a Safari	Morning launcher
osascript -e	App-level control	tell app 'Spotify' to...	Notifications, tabs
shell=True string	Pipes, wildcards	'cat *.log grep ERR'	Quick one-off scripts

 TAKEAWAY

Plain shell for reads. `open` to launch. `osascript` to control. shell=True only when you genuinely need a shell.

Build `os_bridge.py`

A small module that wraps the three patterns from today into reusable functions. You'll import it from every Week 3 build that follows.

FUNCTIONS

`run_shell(cmd)`

- Takes a list of args
- Returns (stdout, returncode)
- `capture_output=True, text=True`
- Print stderr if returncode $\neq$ 0

FUNCTIONS

`launch_app(name)`

- Wraps: `open -a <name>`
- Test: Safari, Spotify, Notes
- Also: `open_url(url)`
- Raises on bad app name

FUNCTIONS

`osa(script)`

- Wraps: `osascript -e <script>`
- Returns `stdout.strip()`
- Build: `notify(title, body)`
- Build: `safari_tab(url)`

Smoke test: `python3 os_bridge.py` → opens Safari, opens duke.edu in a tab, fires a notification.

Four things that will trip you up.

⚠ ALWAYS PASS A LIST

`subprocess.run(["open", "-a", "Safari"])` — not the string `"open -a Safari"` without `shell=True`. The list form avoids quoting headaches and shell injection.

⚠ APP NAMES MUST MATCH

`open -a 'Safari'` works. `open -a 'safari'` doesn't on case-sensitive setups. Use the exact name as it appears in `/Applications`.

💡 APPLESCRIPT QUOTING

AppleScript uses double quotes inside its string. Wrap the whole thing in single quotes in Python: `osa('tell app "Safari" to ...')`.

💡 CHECK THE RETURN CODE

If `returncode != 0`, something failed. Print `stderr` to find out what. Silent failures are the #1 reason scripts "don't work" with no error.

subprocess is the spine of Week 3.

DAY 17

Global hotkeys

pyautogui types and clicks. Combined with osascript, you can bind cmd+shift+1 to a Python action that drives any app.

DAY 18

Autoclicker

Hotkey toggles on/off through subprocess-backed handlers. Decorators + os_bridge meet here.

DAY 20-21

Morning Launcher

The capstone. JSON-configured routines that fire osascript, open URLs, launch apps, notify — all through today's bridge.

Tomorrow: pyautogui + global hotkeys — Python types into other apps and listens for system-wide shortcuts.